

A Recursive Identity for Power Sums: An Alternative to Faulhaber's Formula

Trinsit W.

Independent Researcher

<https://github.com/trinsitw>

May 2026

Abstract

We present a recursive identity for the sum of k -th powers of the first n positive integers, $S(k, n) = \sum_{i=1}^n i^k$. The identity expresses $S(k, n)$ in terms of $S(k-2, n)$ and lower-order sums, providing a constructive alternative to Faulhaber's formula that requires no reference to Bernoulli numbers. We prove the identity, demonstrate its application to derive closed-form polynomials for $k = 1$ through $k = 5$, and describe a Java implementation that computes the closed-form polynomial in n for arbitrary positive integer k , using exact rational arithmetic and exporting results as L^AT_EX markup.

Keywords: power sums, Faulhaber's formula, recursive identity, odd numbers, symbolic computation, exact arithmetic.

1 Introduction

The problem of computing the sum of k -th powers of the first n positive integers,

$$S(k, n) = 1^k + 2^k + 3^k + \cdots + n^k = \sum_{i=1}^n i^k,$$

has a rich history spanning from antiquity to the modern era. The classical result, known as *Faulhaber's formula*, expresses $S(k, n)$ as a degree- $(k + 1)$ polynomial in n using Bernoulli numbers B_j :

$$S(k, n) = \frac{1}{k+1} \sum_{j=0}^k \binom{k+1}{j} B_j n^{k+1-j}.$$

While complete and well-established, its derivation relies on the theory of Bernoulli numbers, typically introduced via generating functions or the Euler–Maclaurin summation formula [1].

This paper presents a recursive alternative discovered independently, motivated by the observation that consecutive odd numbers $(1, 3, 5, \dots, 2i - 1)$ play a natural role in decomposing i^k . The resulting identity expresses $S(k, n)$ in terms of $S(k - 2, n)$ and previously computed lower-order sums, enabling clean recursive computation.

The approach is fully constructive: starting from base cases $S(0, n) = n$ and $S(1, n) = \frac{1}{2}n(n + 1)$, the identity generates exact polynomial expressions for any k . A Java implementation (available at <https://github.com/trinsitw/java>) automates this process for arbitrary k , using exact rational arithmetic throughout and exporting results as L^AT_EX markup [4].

2 The Main Identity

2.1 Statement

Theorem 1 (Recursive Power-Sum Identity). *For all integers $k \geq 2$ and all positive integers n ,*

$$\sum_{i=1}^n i^k = \left(\sum_{i=1}^n (2i - 1) \right) \left(\sum_{i=1}^n i^{k-2} \right) - \sum_{i=1}^n \left((2i - 1) \sum_{j=1}^i (j - 1)^{k-2} \right). \quad (1)$$

Equivalently, letting $A(n) = n^2$ denote the sum of the first n odd numbers,

$$S(k, n) = A(n) \cdot S(k - 2, n) - C(k, n), \quad (2)$$

where the correction term $C(k, n) = \sum_{i=1}^n (2i - 1) S(k - 2, i - 1)$ is expressible recursively through lower-order power sums.

2.2 Proof

We begin by expressing i^2 as the sum of the first i odd numbers:

$$i^2 = \sum_{j=1}^i (2j - 1). \quad (3)$$

Substituting (3) into the definition of $i^k = i^2 \cdot i^{k-2}$:

$$\sum_{i=1}^n i^k = \sum_{i=1}^n (i^2 \cdot i^{k-2}) = \sum_{i=1}^n \left(\sum_{j=1}^i (2j-1) \right) i^{k-2}. \quad (4)$$

Changing the order of summation (the region $1 \leq j \leq i \leq n$ is equivalent to $1 \leq j \leq n$, $j \leq i \leq n$):

$$\sum_{i=1}^n i^k = \sum_{j=1}^n \sum_{i=j}^n (2j-1) i^{k-2}. \quad (5)$$

Factoring out $(2j-1)$, which is independent of i :

$$= \sum_{j=1}^n (2j-1) \left(\sum_{i=j}^n i^{k-2} \right). \quad (6)$$

Rewriting the inner sum as a difference of two sums starting from 1:

$$= \sum_{j=1}^n (2j-1) \left(\sum_{i=1}^n i^{k-2} - \sum_{i=1}^{j-1} i^{k-2} \right). \quad (7)$$

Distributing $(2j-1)$ and separating into two sums. Since $\sum_{i=1}^n i^{k-2}$ does not depend on j , it factors out of the first sum:

$$= \left(\sum_{j=1}^n (2j-1) \right) \left(\sum_{i=1}^n i^{k-2} \right) - \sum_{j=1}^n \left((2j-1) \sum_{i=1}^{j-1} i^{k-2} \right). \quad (8)$$

Finally, swapping the dummy variables i and j in (8) yields the desired identity (1):

$$\sum_{i=1}^n i^k = \left(\sum_{i=1}^n (2i-1) \right) \left(\sum_{i=1}^n i^{k-2} \right) - \sum_{i=1}^n \left((2i-1) \sum_{j=1}^{i-1} j^{k-2} \right). \quad (9)$$

This completes the proof. ■

Remark 2. The correction term $C(k, n)$ involves $S(k-2, i-1)$ evaluated at the variable upper limit $i-1$. In the implementation this is realised by *polynomial composition*: substituting $(i-1)$ for n in the already-computed polynomial $S(k-2, n)$, then summing the resulting polynomial in i over $i = 1, \dots, n$.

2.3 Base Cases

The recursion is grounded by:

$$S(0, n) = n, \quad S(1, n) = \frac{1}{2}n(n+1).$$

For $k = 2$, Theorem 1 with $k-2 = 0$ gives

$$S(2, n) = n^2 \cdot n - \sum_{i=1}^n (2i-1)(i-1) = n^3 - \sum_{i=1}^n (2i^2 - 3i + 1) = \frac{1}{3}n^3 + \frac{1}{2}n^2 + \frac{1}{6}n,$$

matching the classical result.

3 Application: Deriving $S(5, n)$

We apply Theorem 1 with $k = 5$, using the known formulas

$$S(3, n) = \frac{n^2(n+1)^2}{4}.$$

Step 1. Apply (2):

$$S(5, n) = n^2 \cdot S(3, n) - \sum_{i=1}^n (2i-1) S(3, i-1).$$

Step 2. Substitute $S(3, i-1) = \frac{(i-1)^2 i^2}{4}$:

$$\sum_{i=1}^n (2i-1) \cdot \frac{(i-1)^2 i^2}{4} = \frac{1}{4} \sum_{i=1}^n (2i-1)(i^4 - 2i^3 + i^2).$$

Step 3. Expand and collect:

$$\frac{1}{4} \sum_{i=1}^n (2i^5 - 5i^4 + 4i^3 - i^2) = \frac{1}{2} S(5, n) - \frac{5}{4} S(4, n) + S(3, n) - \frac{1}{4} S(2, n).$$

Step 4. Collect the $S(5, n)$ terms on the left and solve:

$$S(5, n) = n^2 \cdot S(3, n) - \frac{1}{2} S(5, n) + \frac{5}{4} S(4, n) - S(3, n) + \frac{1}{4} S(2, n) \implies \quad (10)$$

$$\boxed{S(5, n) = \frac{1}{12} n^2 (n+1)^2 (2n^2 + 2n - 1).} \quad (11)$$

This matches the classical result, confirming Theorem 1.

The derivation in (10)–(11) mirrors exactly what the Java implementation does symbolically: it builds the polynomial identity, gathers like-degree monomials, and solves for the leading coefficient.

4 Computational Implementation

4.1 Architecture

The Java implementation (`AlternativeToFaulhaberFormula.java`) realises the recursive identity as a symbolic computation system. Four classes collaborate:

RationalNumber: Exact rational arithmetic (p/q in lowest terms), supporting the four basic operations without floating-point error.

IndeterminateExponent: A variable–exponent pair, e.g. n^3 or i^2 .

MultivariateMonomial: A rational coefficient times a list of indeterminate–exponent pairs, e.g. $\frac{3}{4} n^2 i^3$. Supports multiplication and single-variable composition.

MultivariatePolynomial: A sum of monomials with operations for addition, subtraction, multiplication, composition, and simplification. Provides `commonDenominatorForm()` for $\text{\LaTeX}$ output.

4.2 Core Algorithm

The central method `sumOfKthPowersOfFirstNPositiveIntegers(k)` is shown below (simplified for clarity):

Listing 1: Core recursive method (simplified)

```

1 public static MultivariatePolynomial
2   sumOfKthPowersOfFirstNPositiveIntegers(int k) {
3
4   // Return memoised result if available
5   if (cache.get(k) != null) return cache.get(k);
6
7   // Build (2i-1) * [S(k-2, n) - S(k-2, i-1)]
8   MultivariatePolynomial Sk2 =
9     sumOfKthPowersOfFirstNPositiveIntegers(k - 2);
10  MultivariatePolynomial Sk2_at_i_minus_1 =
11    Sk2.compose('n',
12      new MultivariatePolynomial(
13        new MultivariateMonomial(ONE, 'i', 1),
14        new MultivariateMonomial(new RationalNumber(-1)))
15      );
16  MultivariatePolynomial oddFactor =
17    new MultivariatePolynomial(
18      new MultivariateMonomial(TWO, 'i', 1),
19      new MultivariateMonomial(MINUS_ONE));
20  MultivariatePolynomial rhs =
21    oddFactor.multiply(Sk2.subtract(Sk2_at_i_minus_1));
22
23  // Sum rhs over i = 1..n (dispatches each monomial
24    // recursively)
25  MultivariatePolynomial summed = sumOfIndexedPolynomial(rhs);
26
27  // Isolate S(k,n): coefficient of i^k moved to lhs
28  MultivariateMonomial kthTerm = /* extract i^k monomial */
29    ...;
30  MultivariatePolynomial formula =
31    sumOfIndexedPolynomial(rhs.minus(kthTerm))
32    .multiply(ONE.divide(ONE.subtract(kthTerm.coefficient())))
33    );
34
35  cache.put(k, formula);
36  return formula;
37 }

```

The key operations are:

1. **Compose:** substitute $(i - 1)$ for n in $S(k - 2, n)$, obtaining a polynomial in i .
2. **Sum indexed polynomial:** for each monomial $c \cdot i^j$ in the expanded expression, replace it with $c \cdot S(j, n)$ (recurring into the cache).
3. **Solve:** gather all $S(k, n)$ terms (monomials in i^k) to the left-hand side and divide through.

4.3 Output

Running `main()` produces L^AT_EX-formatted polynomial expressions for $k = 1$ through $k = 100$. A representative excerpt:

```
\Sigma_{i^5} = \frac{1}{12}n^2(n+1)^2(2n^2+2n-1)
\Sigma_{i^6} = \frac{1}{42}n(n+1)(2n+1)(3n^4+6n^3-3n+1)
```

All outputs were verified against tabulated Faulhaber results for $k \leq 10$.

5 Comparison with Faulhaber's Formula

Table 1: Comparison of the two approaches

Aspect	Faulhaber's formula	Present identity
Prerequisites	Bernoulli numbers	Elementary algebra only
Form	Direct closed form	Recursive
Computation	Evaluate one formula	Memoised recursion
Structural insight	Hidden in Bernoulli numbers	Explicit odd-number decomposition
Rational coefficients	Yes (via B_j)	Yes (exact arithmetic)

The present identity reveals a direct relationship between i^k and the odd numbers $(2i - 1)$, not apparent from the Bernoulli-based approach. Specifically, equation (2) shows that $S(k, n)$ is a correction to $n^2 S(k - 2, n)$, connecting successive even- k power sums via the square-number identity $\sum_{i=1}^n (2i - 1) = n^2$.

6 Sample Output

The following polynomial formulas are generated by the implementation for $k = 1, \dots, 8$:

$$\begin{aligned}
S(1, n) &= \frac{1}{2}n^2 + \frac{1}{2}n \\
S(2, n) &= \frac{1}{3}n^3 + \frac{1}{2}n^2 + \frac{1}{6}n \\
S(3, n) &= \frac{1}{4}n^4 + \frac{1}{2}n^3 + \frac{1}{4}n^2 \\
S(4, n) &= \frac{1}{5}n^5 + \frac{1}{2}n^4 + \frac{1}{3}n^3 - \frac{1}{30}n \\
S(5, n) &= \frac{1}{6}n^6 + \frac{1}{2}n^5 + \frac{5}{12}n^4 - \frac{1}{12}n^2 \\
S(6, n) &= \frac{1}{7}n^7 + \frac{1}{2}n^6 + \frac{1}{2}n^5 - \frac{1}{6}n^3 + \frac{1}{42}n \\
S(7, n) &= \frac{1}{8}n^8 + \frac{1}{2}n^7 + \frac{7}{12}n^6 - \frac{7}{24}n^4 + \frac{1}{12}n^2 \\
S(8, n) &= \frac{1}{9}n^9 + \frac{1}{2}n^8 + \frac{2}{3}n^7 - \frac{7}{15}n^5 + \frac{2}{9}n^3 - \frac{1}{30}n
\end{aligned}$$

These match the classical Faulhaber results exactly.

7 Conclusion

We have presented a recursive identity for power sums that provides an elementary alternative to Faulhaber’s formula. The identity is self-contained, requiring only basic algebraic manipulation and induction, and is directly amenable to symbolic computation.

The accompanying Java implementation demonstrates that the identity can be automated to produce exact closed-form polynomial expressions for $S(k, n)$ for arbitrary positive integer k , with exact rational coefficients and L^AT_EX output.

Future directions include:

1. A formal analysis of the coefficient structure and its relation to Bernoulli numbers.
2. Extension to weighted power sums $\sum_{i=1}^n w(i) i^k$ for polynomial weights w .
3. A proof that the recursion always terminates with a polynomial of the correct degree $k + 1$ in n .

Acknowledgment

The mathematical ideas, recursive identity, and Java implementation in this paper are entirely the author’s own work. AI assistance (Claude, Anthropic) was used for manuscript preparation and L^AT_EX typesetting.

References

- [1] D. E. Knuth, *Johann Faulhaber and Sums of Powers*, Mathematics of Computation **61**(203), 277–294, 1993.
- [2] B. C. Berndt, *Ramanujan’s Notebooks, Part II*, Springer-Verlag, New York, 1989.
- [3] J. H. Conway and R. K. Guy, *The Book of Numbers*, Copernicus/Springer, New York, 1996.
- [4] Trinsit W., *AlternativeToFaulhaberFormula.java*, GitHub repository, 2026. <https://github.com/trinsitw/java>